

CMT2310A Quick Start Guide

Overview

This application document provides key functions and introduction on CMT2310A product development as the purpose of helping users quickly start application of the product.

Part number covered under this document is shown as follow:

Table 1. Part Number Covered in this Document

Part Number	Frequency	Modem	Function	Configuration	Package
CMT2310A	113 -960MHz	(4)(G)FSK/OOK	Transceiver	Register	QFN24

Users should jointly read the documents below to get comprehensive information to assist software and hardware development.

AN238 CMT2310A RF Parameter Configuration Guide

AN235 CMT2310A FIFO and Package Format Usage Guide

AN236 CMT2310A Register Description

AN239 CMT2310A Automatic Receiving and Transmission Function Usage Guide

Table of Contents

1	Chip Introduction	3
1.1	Brief Introduction	3
1.2	IO Pin Description	4
2	SPI Interface Timing	5
2.1	Read / Write Register Operation	5
2.2	Burst Read / Write Operation	6
2.3	Read/Write FIFO Operation	8
3	Configuration and Control Mechanism	10
3.1	Register Overview	10
3.2	Operation State Switching	11
3.3	Chip Reset Mechanism	14
3.4	RFPDK Introduction	14
3.5	Chip Initialization Process	20
3.6	Procedure Summary	20
4	cmt2310a_easy_demo Introduction	21
4.1	Software Hierarchy Architecture	21
4.2	Software Implementation and Calling Relation	22
4.2.1	CMT2310A Initialization	23
4.2.2	CMT2310A Configuration	24
4.2.3	CMT2310A State Processing	25
4.3	Software Directory Structure	27
4.3.1	Application Layer Source Code	28
4.3.2	Simulating SPI Source Code	29
4.3.3	Abstract Hardware Layer Source Code	29
4.3.4	Chip Driver Layer Source Code	32
4.3.5	Chip Processing Layer Source Code	33
5	Appendix	34
5.1	Appendix 1 : SPI Read/Write Sample Code	34
5.2	Appendix 2 : State Switching Library Function Sample Code	40
5.3	Appendix 3 : Initialization Function Sample Code	43
6	Revise History	45
7	Contacts	46

1 Chip Introduction

1.1 Brief Introduction

CMT2310A is a digital and analog integrated transceiver. It uses the 32 MHz crystal to provide the PLL reference frequency and the digital clock as well as the supporting of the OOK, 2(G)FSK and 4(G)FSK modulation/demodulation modes. Meanwhile, it also supports both Direct and Packet data processing modes.

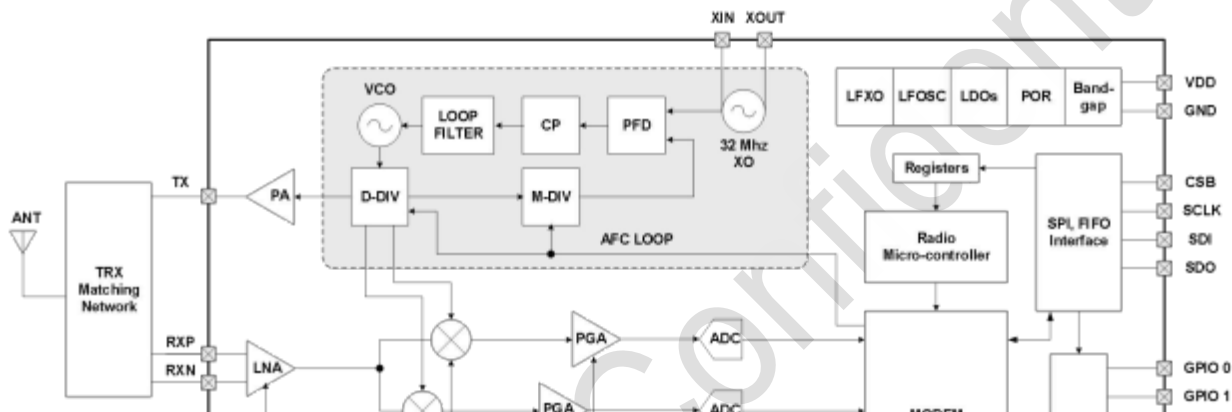


Figure 1. CMT2310A System Block Diagram

The chip adopts low-IF structure of LNA + PGA + ADC + PLL to realize the Sub- 1 GHz wireless reception function and PLL + PA structure to realize Sub- 1 GHz wireless transmission function.

In the receiver system, the analog circuit mixes the RF signal to IF and converts the signal from analog to digital through the ADC module, then outputs I/Q two single bit signals to the digital circuit for further digital demodulation. The digital circuit is responsible for mixing the intermediate frequency to zero frequency (Base band) and performing a series of filtering and decision processing, as well as AFC and AGC dynamically control of the analog circuit , and finally demodulated the original 1-bit signal. After demodulation, the signal will be sent to the decoder to decode and then be filled in the FIFO, or output to GPIO directly.

In the transmitter system, the digital circuit will encode and packet the data and then send to the modulator (or to the modulator directly without encoding and packet). The modulator will directly control the PLL and PA, modulate the data by (G) FSK or OOK and transmit.

The chip contains an internal mini-microcontroller (support for internal factory programming only), which is responsible for scheduling various chip internal operations including enriched wireless transmission and receiving processing such as Auto-ACK, automatically frequency skipping, DUTY-CYCLE low-power transmission and receiving, CSMA, etc.

The chip provides the SPI communication port and the external MCU can configure the various functions by accessing registers, control the main state machine, and access FIFO.

1.2 IO Pin Description

Take the QFN24 package as an example to introduce the CMT2310A pin arrangement and function:

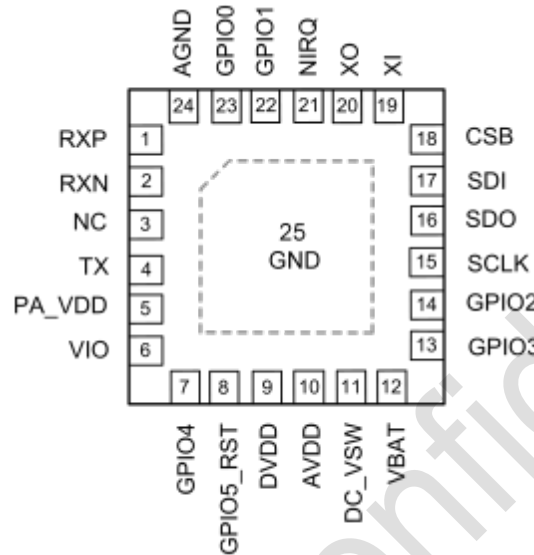


Figure 2. CMT2310A Pin Diagram

Table 2. CMT2310A Pin Description

Pin Number	Name	I/O	Description
1	RXP	I	RF signal input P
2	RXN	I	RF signal input N
3	NC	--	Hanging
4	TX	O	PA output
5	PA_VDD	IO	PA VDD
6	VIO	IO	IO VDD
7	GPIO4	IO	It can be configured as: DOUT, INT1, INT2, DCLK, CLKO, LFCLKO.
8	GPIO5_RST	IO	It can be configured as: RSTn, INT1, INT2, DOUT, DCLK.
9	DVDD	I	Digital VDD
10	AVDD	I	Analog VDD
11	DC_VSW	I	DCDC
12	VBAT	I	Analog VDD
13	GPIO3	IO	It can be configured as: INT1, INT2, DCLK, DOUT, LFXO2, ANTD2.
14	GPIO2	IO	It can be configured as: INT1, INT2, INT3, DCLK, DOUT, LFXO1, ANTD1.

Pin Number	Name	I/O	Description
15	SCLK	I	SPI Clock
16	SDO	O	SPI data output
17	SDI	I	SPI data input
18	CSB	I	SPI chip selected
19	XI	I	Crystal circuit input.
20	XO	O	Crystal circuit output.
21	NIRQ	I	It can be configured as: INT1, INT2, DCLK, DOUT, TCXO.
22	GPIO1	IO	It can be configured as: DCLK, INT1, INT2, DOUT, TRX SWT.
23	GPIO0	IO	It can be configured as: DOUT, INT1, INT2, INT3, DCLK, TRX_SWT.
24	AGND	I	Analog GND
25	GND	I	Bottom GND

2 SPI Interface Timing

The chip communicates with outside through the 4-wire SPI interface. The SPI is 4-wire by default and it can be configured as 3-wire after power. The active low CSB is the chip selected signal for register access. The SCLK is series port clock with a maximum speed up to 10 MHz. Both of the chip and external MCU send out data on SCLK's falling edge and collect data on the rising edge. The SDI is for data input and SDO for data output. If it is under 3-wire mode, SDI is used for both data input and output while SDO leave idle. Both address and data are sent out from MSB.

2.1 Read / Write Register Operation

When accessing registers, it pulls CSB low first then sends a R/W bit followed by a 7-bit register address. After the external MCU pulls down the CSB, it must wait for at least half a SCLK cycle, and then send the R/W bit. After the MCU sends out the last falling edge of SCLK, it must wait for at least half a SCLK cycle, and then pulls the CSB high.

Note that for the 4-wire write register operation, while the SDI enters the write data, the SDO outputs the current register value (old register read data), and the MCU can decide whether to read the value as needed.

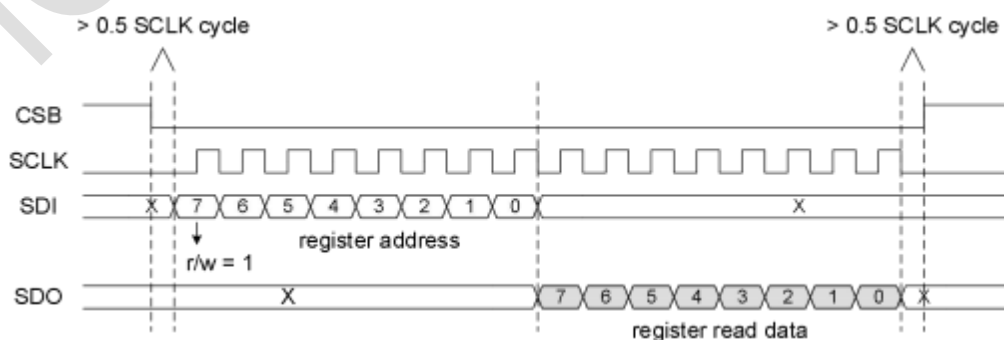


Figure 3. SPI (4-wire) Register Read Timing

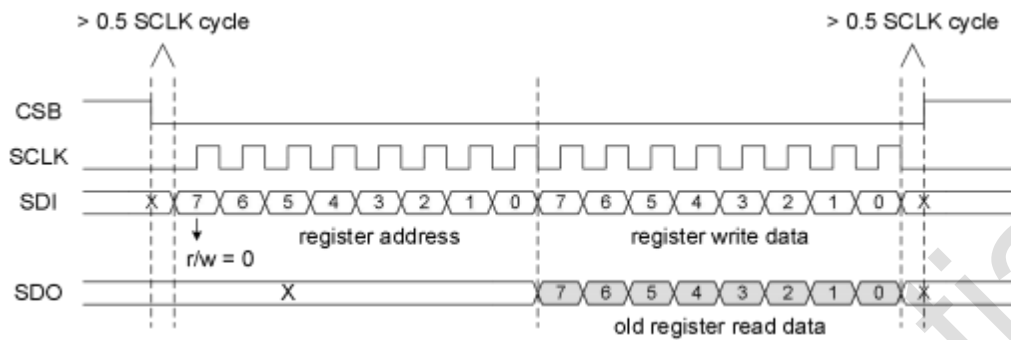


Figure 4. SPI (4-wire) Register Write Timing

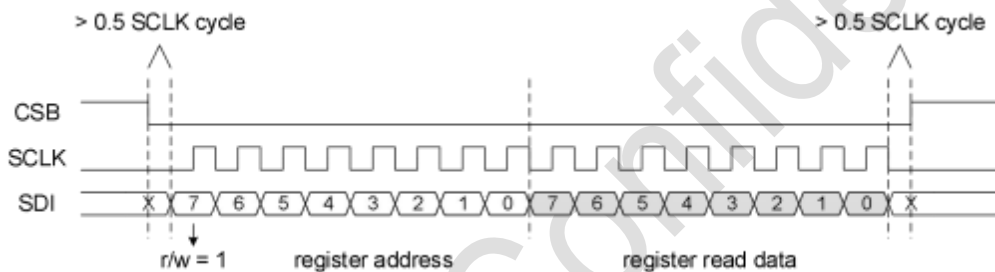


Figure 5. SPI (3-wire) Register Read Timing

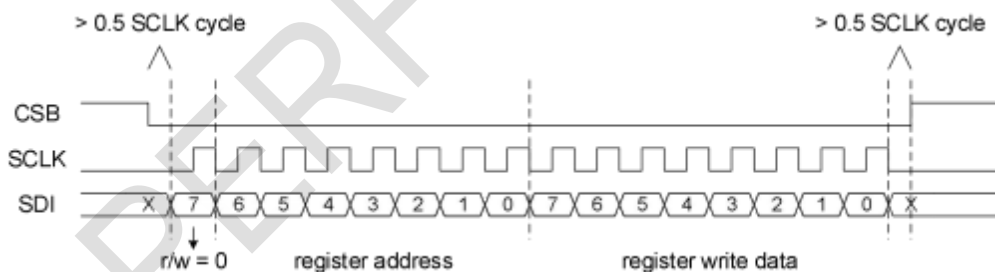


Figure 6. SPI (3-wire) Register Write Timing

For the register read operation in Figure 6, both MCU and CMT2310A will switch IO (SDI) between address 0 and data 7. For example, CMT2310A switches IO port from input to output while the MCU will switch IO port from output to input. Please notice the dot line position in middle; It is highly recommended that the MCU switch the IO port to the input before sending out the falling edge of the SCLK. The CMT2310A switches IO to input after the falling edge. So that it can prevent potential electrical conflict as well as some MCU reset or abnormal behavior caused by both sides when setting SDIO as output at the same time.

For the SPI read-write register operation sample code please see in Appendix 1.

2.2 Burst Read / Write Operation

Except for the single-byte read/write register operations mentioned above, the SPI can also support read/write registers in a batch mode. The BURST read-write operation is triggered by writing 0x7B address BRW_PORT of Page 0. When r/w bit is 0, the write register performed and if it is 1, the read register performed.

The Burst read/write operation can also use 3-wire SPI. When 3-wire is used, both the output of read data and input of write data are conducted on SDI port. When a 4-wire is used, the written data is input from SDI and the read data is output from SDO. The Burst operating procedure is: access address 0x7B of BRW operating port first; the r/w bit contained in the data is used to determine whether it is read or write operation; After the continuous read or written data part, users decide when to complete the operation by themselves.

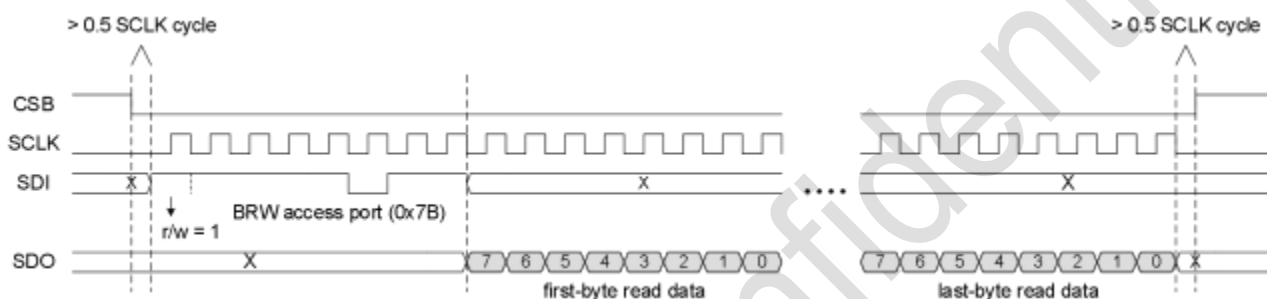


Figure 7. SPI (4-wire) Burst Read Timing

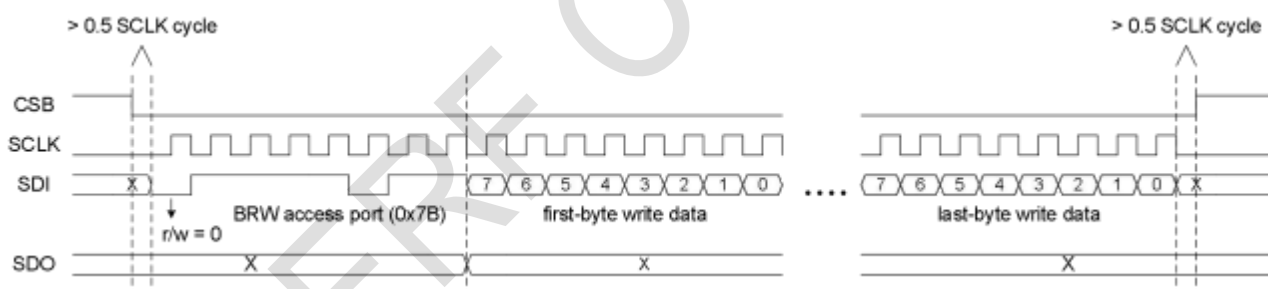


Figure 8. SPI (4-wire) Burst Write Timing

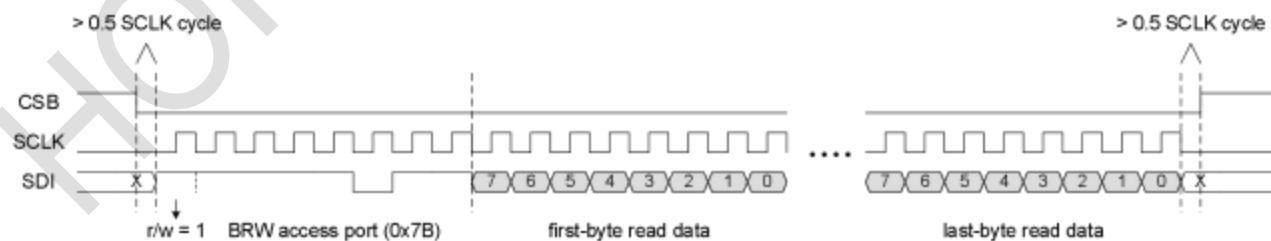
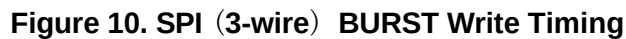


Figure 9. SPI (3-wire) BURST Read Timing



The maximum range of addresses available for BURST read and write for each Page is listed as followed:

Users can select any start address within the range of each Page to conduct a continuous BURST read and write operation of N addresses. While value of N has to make sure that the ending address cannot exceed the address range. The common way of BURST reading and writing is to let users make a quick one-time configuration before calling. The configuration register content comes from the RFPDK tool and the content of the skipping frequency table is designed by users themselves.

CMT2310A provides two separated 128-byte FIFO by default, which are used for RX and TX. RX FIFO is used for store received data in Rx mode while TX FIFO is for store data transmitted in Tx mode. Users can also set FIFO_MERGE_EN to 1 to merge the two separated FIFO into one 256-byte FIFO, which can be used in both TX and RX mode. By configuring FIFO_RX_TX_SEL, it can indicate whether it is currently used as TX FIFO or RX FIFO. When the two FIFO are not merged, users can fill the next time 128-byte TX FIFO when the 128-byte RX FIFO filled in simultaneously to save the system operation time.

The FIFO can be accessed via the SPI interface. Users can empty the FIFO by setting the TX_FIFO_CLR / RX_FIFO_CLR bit. Users can repeatedly filled data before transmission by setting FIFO_RESTORE without refill data.

Users can control whether FIFO still store data in sleep mode by configuring PD_FIFO, that is, PD_FIFO = 0 stands for FIFO storing data in sleep mode, yet with 200 nA leakage current consuming.

When the MCU needs to access FIFO, some registers need to be configured to set the read / write mode of FIFO and other working modes. See AN235 CMT2310A FIFO and Package Format Use Guide, which will show the read and write timing diagram after the pattern is determined. FIFO is triggered by writing address 0x7A of Page 0, write FIFO will be opened when r / w bit is 0, and read FIFO on the contrary, will on when it is 1.

The FIFO read and write can also be operated through 3-wire SPI; when it is 3-wire, the read data output and write data input is done on the SDI pin. When using 4-wire, write data is input from SDI and read data is output from SDO. The operation process of FIFO is to access the 0x7A address of FIFO operation port first, which will determine whether to write or read data later. Users decide when to stop the operation at the continuous read or written data situation.

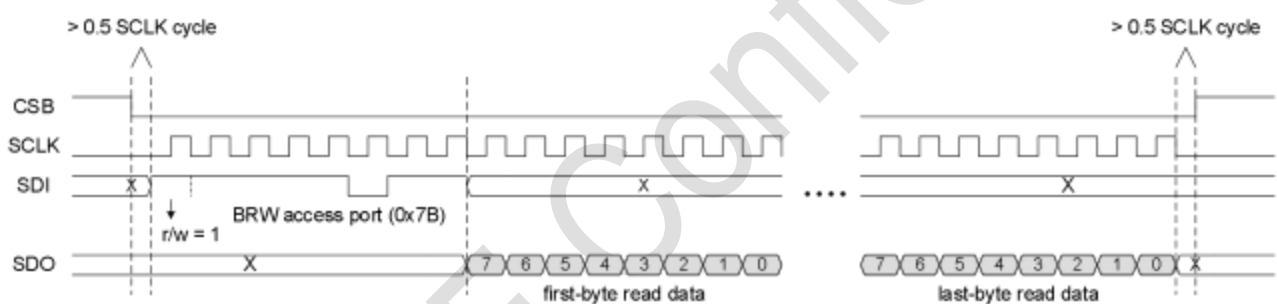


Figure 11. SPI (4-wire) Read FIFO Timing

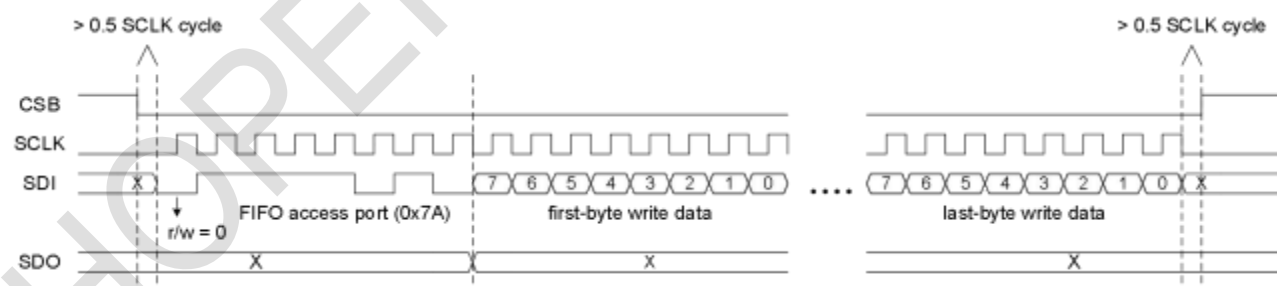


Figure 12. SPI (4-wire) Write FIFO Timing

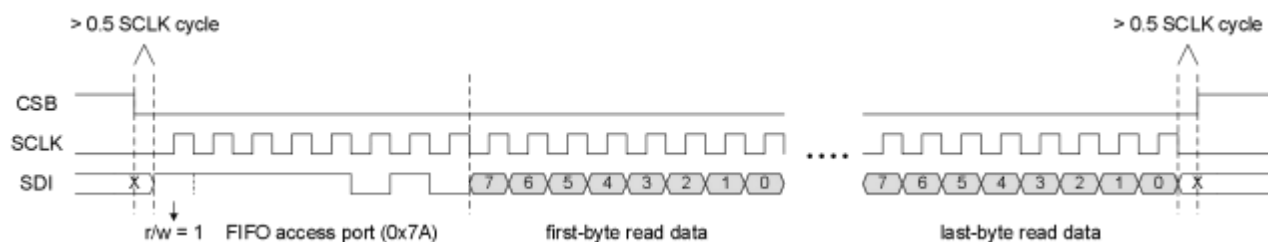


Figure 13. SPI (3-wire) Read FIFO Timing

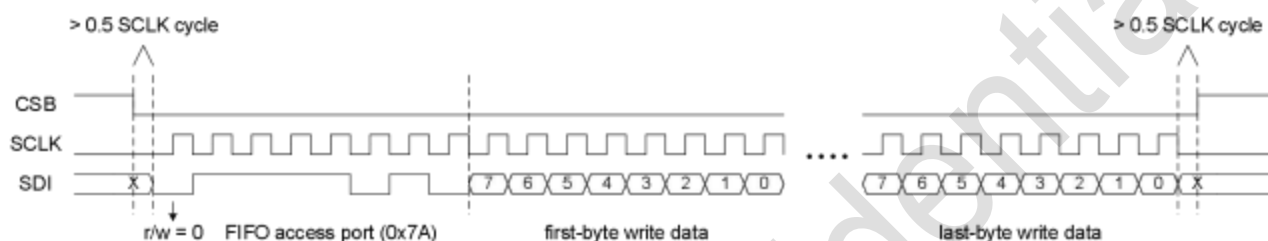


Figure 14. SPI (3-wire) Write FIFO Timing

For the SPI FIFO read and write operation code, please take reference in appendix 2.

3 Configuration and Control Mechanism

3.1 Register Overview

The external MCU configures and controls the chip by accessing the registers in 3 banks through the SPI interface. Switching among 3 banks can be fulfilled by writing address 0x7E as shown in the table below. The address from 0x00 to 0x7F can be divided into 2 major banks, namely, configuration bank (including 6 sub-banks), control bank (including 2 sub-banks).

In fact, there is no difference in operation mode between 2 banks for their continuous addresses, both of which are read and write by accessing registers through the SPI. However, the 2 banks still work differently according to function and using way. More information are shown in the table below.

Table 3. CMT2310A Register Partition Table

Page	Address	Function Area	Function Description
0	0x00 - 0x17	System control area	It is used for chip control, such as state switching, state inquiry as well as some other enable chip functions.
	0x18 - 0x27	Interrupt control area	It is used for reading and clearing interrupt flags while inaccessible in sleep state.
	0x28 - 0x5F	Configuration area (packet format)	It is used for packet format and FIFO related functions.

	0x60 - 0x77	Configuration area (system feature)	It is used for configure the system working-related functions.
1	0x00 - 0x0F	Configuration area (CMT specific)	It is for CMT internal use.
	0x10 - 0x27	Configuration area (TX)	It is used for configuring transmitter features.
	0x30 - 0x68	Configuration area (RX)	It is used for configuring receiver features.
2	0x00 - 0x3F	Configuration area (auto-hopping)	It is used for storage of auto-hopping channel list.

Notes:

1. The 0x7A address is the port address of the FIFO read and write in page 0,
2. The 0x7B address is the port address of the register batch read and write in all pages.
3. The 0x7E address is the page switch address in all pages.
4. The 0x7F address is soft reset address in all pages.
5. All other addresses unlisted above are inaccessible.

All remaining areas can be accessed in sleep state except for the interrupt configuration area. As long as there's no battery power losses, the chip will not conduct POR reset and the configured content will stay complete.

The register values in pages 0 and 1 can be either comes from the RFPDK or changed by the customer during the application. In general, except for individual RF frequency or data rate configurations, which may need to be configured multiple times in the application, most other registers only need to be configured once during the initialization process.

As for the hopping frequency list in page 2, users need to design and configure according to the AN documents.

3.2 Operation State Switching

The working state switching diagram of the chip is shown in the following figure. There are 7 states in which users can send state switching command through configuration of CTL_REG_0 and CTL_REG_1 registers and they can query the corresponding code value of each state by reading the CTL_REG_10 register.

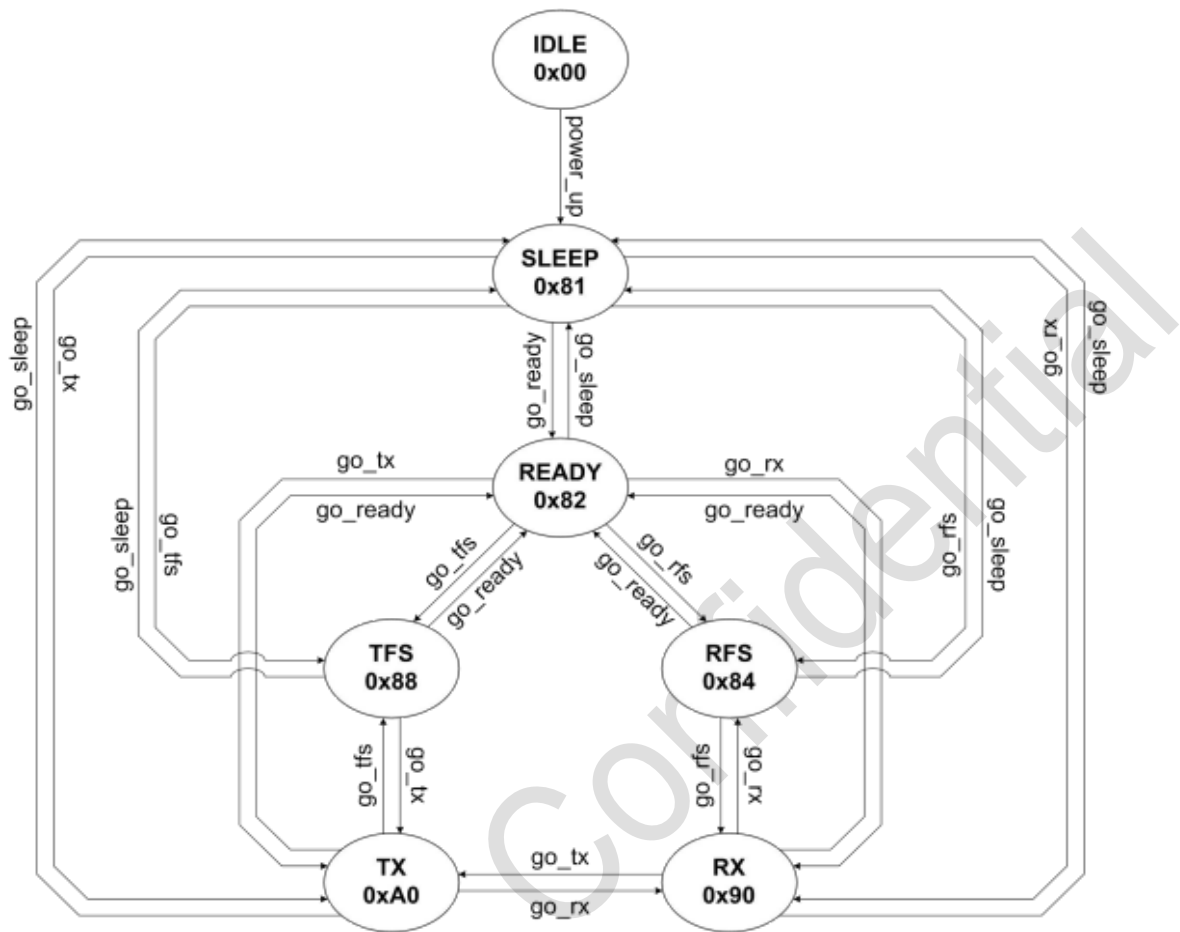


Figure 15. CMT2310A State Switching Diagram

Table 4. CMT2310A Status and Enable Modules

Status	Status Code	Switching Command	Enabled Module	Selected Enabled Module
IDLE	0x00	soft_rst	SPI, POR	None
SLEEP	0x81	go_sleep	SPI, POR	LFOSC/LFXO, FIFO, Sleep Timer
READY	0x82	go_ready	SPI, POR, XTAL, FIFO	None
RFS	0x84	go_rfs	SPI, POR, XTAL, PLL, FIFO	None
TFS	0x88	go_tfs	SPI, POR, XTAL, PLL, FIFO	None
RX	0x90	go_rx	SPI, POR, XTAL, PLL, LNA+MIXER+ADC, FIFO	RX Timer
TX	0xA0	go_tx	SPI, POR, XTAL, PLL, PA, FIFO	None

The external MCU can switch and query the chip operating status by accessing the following control zone registers.

Table 5. State Switching Register

Register Name	Bit	R/W	Bit Flag	Function Description
CTL_REG_00 (0x00)	7:0	W	POWERUP<7:0>	Power up command: 0x03: power_up Other values are not allowed to fill in.
CTL_REG_01 (0x01)	7:0	W	CHIP_MODE_SWT<7:0>	State switching command: 0x01: go_sleep 0x02: go_ready 0x20: go_rfs 0x10: go_tfs 0x08: go_rx 0x04: go_tx Other values are not allowed to fill in.
CTL_REG_10 (0x0A)	7:0	R	CHIP_MODE_STA<3:0>	Chip status code: 0x00: IDLE 0x81: SLEEP 0x82: READY 0x84: RFS 0x88: TFS 0x90: RX 0xA0: TX Other values: ineffective If other values received, it indicates that the chip working abnormally and it is recommended to reset the chip.

The chip sometimes needs to wait for a certain time to switch the state successfully after the MCU sends the go_* command. Each state time and the required switching waiting time will be described respectively as follow .

● IDLE State and Power-up Flow

After the chip VDD is powered up, it usually needs to wait about 1 ms to release POR. However, the chip will remains in IDLE status with no actions. After the user sends the power_up command, the chip starts the power-up process and corrects each module. The chip stays at the SLEEP after completing the correction. The chip will back to IDLE status, waiting for the new power_up command so long as it needs to do any reset. (including POR, hardware reset and software reset)

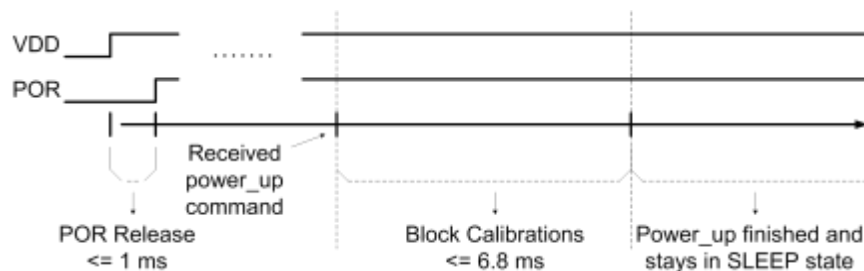


Figure 16. CMT2310A Power-up Diagram

The following table lists the required state switching time, and starting state is at the far left .

Table 6. State Switching Time

Source State	Target State					
	SLEEP	READY	RFS	RX	TFS	TX
SLEEP		660 us	770 us	820 us	770 us	820 us
READY	Immediate		110 us	160 us	110 us	160 us
RFS	Immediate	Immediate		20 us	Non- switchable	Non- switchable
RX	Immediate	Immediate	Immediate		Non- switchable	160 us
TFS	Immediate	Immediate	Non- switchable	Non- switchable		20 us
TX	Immediate	Immediate	Non- switchable	160 us	immediate	

Notes:

In Direct mode, if the chip is transmitting, it will exit the TX state immediately once it receive the switching command. .

In Packet mode, if the chip is transmitting, it will exit the TX state after finish the current transmission.

3.3 Chip Reset Mechanism

The CMT2310A supports 3 types of reset, power-up reset (POR), hardware reset (GPIO5) and software reset (SOFTTRST).

As the name suggests, power reset means the chip reset when it is power-on. Hard reset is achieved by pulling the GPIO 5 up, and when the hard reset takes effect and released, the chip returns to the IDLE state. Software reset is achieved by writing 0xFF to the address 0x7F by the SPI. Once the chip receives the command, the chip will immediately reset and back to the IDLE state. However, software reset will not reset configuration registers in Page 0, 1,2 as power-up reset and hardware reset will reset all the registers.

3.4 RFPDK Introduction

RFPDK is a software tool provided by CMOSTEK for configuring or burning RF chips. It has to install and run in the Windows environment. For the CMT2310A, the role of RFPDK is to generate the register profiles through the parameters that users introduce on the interface. Users can import the file into the MCU for the register configuration.

The CMT2310A configuration mechanism is: users first generate the register configuration file through RFPDK to initial the chip and then users can flexibly configure and control some of the specific registers according to the applications. Therefore, users only need to know how to operate the RFPDK, and selectively understand some of the registers which is need to know.

The previous chapters have briefly introduced the registers. So in this section, we will show a brief introduction to the RFPDK and at the subsequent sections we will introduce how to combine the two configuration.

The main interface of RFPDK is shown as follow:

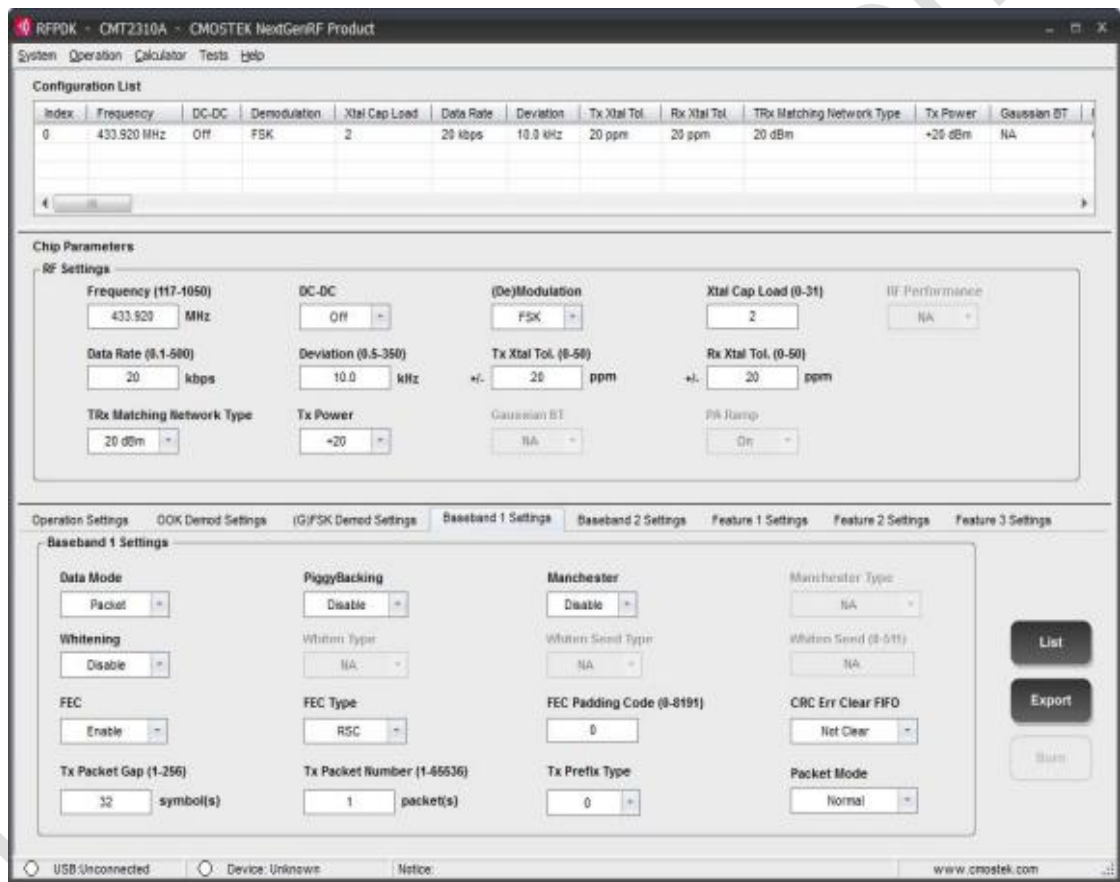


Figure 17. CMT2310A RFPDK

Upon the interface, there are 9 blocks need to input by the users:

Table 7. Main Block of RFPDK

RFPDK Sections	Configuration Parameters	RFPDK Input Parameter Vs. Related Registers
RF Settings	Frequency, data rate, Tx power and other RF parameters	Register contents comes from the complex calculation of input parameters. Users do not need to know the contents.
Operation Settings	System operating parameters	Input parameters match with the corresponding registers. It is easily understood by users.
OOK Demod Settings	OOK demodulation parameters	Register contents comes from the complex calculation of input parameters. Users do not need to know the contents. Normally, they can use the defaulted parameters.
FSK Demod Settings	FSK demodulation parameters	Register contents comes from the complex calculation of input parameters. Users do not need to know the contents. Normally, they can use the defaulted parameters.
Baseband 1 Settings	Baseband code and FIFO parameters	Input parameters match with the corresponding registers. It is easily understood by users.

RFPDK Sections	Configuration Parameters	RFPDK Input Parameter Vs. Related Registers
Baseband 2 Settings	Baseband packet format parameters	Input parameters match with the corresponding registers. It is easily understood by users.
Feature 1 Settings	Other function parameters	Input parameters match with the corresponding registers. It is easily understood by users.
Feature 2 Settings	Other function parameters	Input parameters match with the corresponding registers. It is easily understood by users.
Feature 3 Settings	Other function parameters	Input parameters match with the corresponding registers. It is easily understood by users.

The following is an example of generating configuration file.

The upper part of the file is the comment section to confirm the current successfully generated configuration information, which is usually used for error checking. Users don't need to pay too much attention to it.

```

;-----
;   CMT2310A Configuration File
;   Generated by CMOSTEK RFPDK 1.53_Beta_15
;   2022 .01. 18 16:54
;-----
; Mode                      = Advanced
; Part Number               = CMT2310A
; Frequency                 = 433.920 MHz
; DC-DC                    = Off
; Demodulation              = FSK
; Xtal Cap Load            = 2
; Data Rate                 = 20.0 kbps
; Deviation                 = 10.0 kHz
; Tx Xtal Tol.              = 20 ppm
; Rx Xtal Tol.              = 20 ppm
; TRx Matching Network Type = 20 dBm
; Tx Power                  = +20 dBm
; Gaussian BT               = NA
; PA Ramp                   = On
; RF Performance            = NA
; Rx Duty-Cycle             = Off
; Tx Duty-Cycle             = Off
; Sleep Timer               = Off
; Sleep Time                = NA
; Rx Timer                  = Off
; Rx Time T1                = NA
; Rx Time T2                = NA
; Rx Exit State             = STBY
; Tx Exit State             = STBY
; TX Duty Cycle Persist     = Off
; Packet Done Exit          = On
; TX Duty-Cycle Times       = 0
; SLP Mode                  = Mode 0
; RSSI Valid Source
; PJD Window

```

	= RSSI Compare
	= NA
; RSSI Compare TH	= - 127 dBm
; CDR Type	= Counting
; AFC	= autoselect
; FSK2 Data Map	= 0:F-low 1:F-high
; FSK4 Data Map	= NA
; CDR Type	= Counting
; Channel BW	= autoselect
; Baseband BW FSK	= autoselect
; Data Mode	= Packet
; Packet Mode	= Normal
; PiggyBacking	= Disable
; Manchester	= Disable
; Manchester Type	= NA
; Whitening	= Disable
; Whiten Type	= NA
; Whiten Seed Type	= NA
; Whiten Seed	= NA
; FEC	= Disable
; FEC Type	= NA
; FEC Padding Code	= NA
; crc err clear fifo	= Not Clear
; Tx Packet Gap	= 32 symbol(s)
; Tx Packet Number	= 1 packet(s)
; Tx Prefix Type	= 0
; Packet Type	= Fixed Length
; Address-Length Position	= NA
; Length Size	= 1-byte
; Payload Bit Order	= Start from msb
; Address Field	= Disable
; Preamble Rx Size	= 2
; Preamble Tx Size	= 8
; Preamble Value	= 170
; Preamble Unit	= 8-bit
; Sync Size	= 3-byte
; Sync Format	= S2LP
; Sync Value	= 3003605
; Sync Manchester	= Disable
; Sync Value Selection	= Sync Value
; Sync FEC Value	= 3003605
; Sync Tolerance	= None
; Address Detect Mode	= None
; Address Split Mode	= NA
; Address Size	= NA
; Address Err Mask	= NA
; Address Free	= NA
; Dest Addr Value	= NA
; Src Addr Value	= NA
; Dest Addr Bit Mask	= NA
; Src Addr Bit Mask	= NA
; Sequence Num	= None
; Sequence Num Match	= off
; Sequence Num Mode	= NA
; Sequence Num Value	= NA
; FCS2	= None
; FCS2 Value	= NA

~~; Payload Length~~

; CRC Options
; CRC Swap

= 32
= None
= NA

; CRC Seed
; CRC Bit Invert
; CRC Range
; CRC Polynomial
; CRC Bit Order
; CRC Refin
; CRC_Refout
; Frequency Hopping Mode ; Freq Hopping Space ; Hopping Channels ; CSMA Mode
; CSMA RSSI Detection
; Hopping Persist
; Hopping Intermediate State ; CSMA Sleep Timer Random ; CSMA Rx Time
; CSMA Sleep Time M
; CSMA Sleep Time R
; CSMA Persist
; CSMA Detect Times ; Tx Auto Hopping ; Rx Auto Hopping ; Auto Acknowledge
; Auto Resend ; Maximum Resend Times ; RSSI Detect Mode ; LFOSC LFXO Sel
; LF Clock Out
; Dout Mute
; Dout Mute Sel
; dout adjust mode
; Dout Adjust Percentage
; LBD Threshold
; Antenna Diversity
; Antenna Switch Mode
; Collision Detect
; Collision Step
; RSSI Offset dB
; RSSI Offset Sel
= NA
= NA
= NA
= NA
= NA
= NA
= Mode 2
= 10 kHz
= 10
= Disable
= NA
= Disable
= TRFS
= NA
= NA
= NA
= NA
= NA
= NA
= Disable
= Disable
= off
= off
= 1
= always

= LFOSC(32kHz)
= off
= disable
= NA
= disable
= NA
= 2.4 v
= off
= NA
= off
= NA
= NA
= autosel

The second half of the file is the register configuration content that the user needs to import into the MCU program, recording in hexadecimal. The tool will automatically divide the contents of all registers into 3 pages, covering the configuration area of registers on page 0, 1 and 2 for the convenient of users.

3.5 Chip Initialization Process

Users need to use the RFPDK to generate the content of the configuration area for the MCU for the initial configuration. After the chip is powered up or reset, it will automatically enter into SLEEP to wait for MCU operation. The MCU can be operated by the following initialization procedure:

1. It sends a hardware or software reset to CMT2310A when MCU gets ready. If the hardware reset on GPIO5 is not used, GPIO5_SEL is configured to 101 to make GPIO 5 keep on outputting low level after software reset is conducted.
2. Query register CHIP_MODE_STA<7:0> to confirm the chip stays in IDLE state (0x00).
3. Send power_up command to make the chip power up, then after wait for 10 ms, confirm the chip enters SLEEP state (0x81)
4. Write the register content generated by RFPDK into the configuration areas of Page 0 and 1.
5. If auto-frequency-hopping is required, write the designed frequency-hopping table into the configuration area of Page 2.

Once the initialization is complete, MCU can start to work. The chip needs to safely switch and stay in SLEEP or READY status at anytime when there is a need to change all or part of the configuration registers.

3.6 Procedure Summary

According to the above information, there are 3 steps for users to operate the chip:

1. Read the relevant AN documents and use RFPDK to generate the wanted register file.
2. Initialize the chip configuration.

3. Execute the application program. There are two major operations in the application program.

- a) Modify the configuration bank — specify which function area is the modified parameter belongs to; generate a new configuration table with RFPDK; pick up the content in the modified bank and import into the program; at the required program position, write the content into the corresponding register address.
- b) Control the chip operation — specify the register meanings of both clear control area and interrupt control area. Be familiar with each chip characteristics and operating rules in AN document, including the FIFO control, IO and interrupt control, RSSI read, auto-frequency hopping, etc; What's more, operate the chip to meet the application requirements.

4 cmt2310a_easy_demo Introduction

This section discusses CMT2310A Demo program, which is closely associated with the various operations discussed in this document. For the functions called in the program, the corresponding source code is provided in Section 5.

4.1 Software Hierarchy Architecture

In order to standardize the CMT2310A operation process and enhance the portability, the Demo program has arranged a hierarchy structure with each module calling the corresponding API function. The whole program is divided into 5 layers as listed below.

1. Application: Application layer, simply sending and receiving data packets in Demo.
2. Radio handlers: Chip processing layer, including the chip initialization, configuration, state control as well as other process.
3. CMT2310A drivers: Chip driver layer, called by the upper layer.
4. Hardware abstraction layer: Hardware abstraction layer, access and control the chip register, FIFO and GPIO.
5. Hardware: Hardware layer, including resources of LED, press button as well as SPI communications provided by MCU.

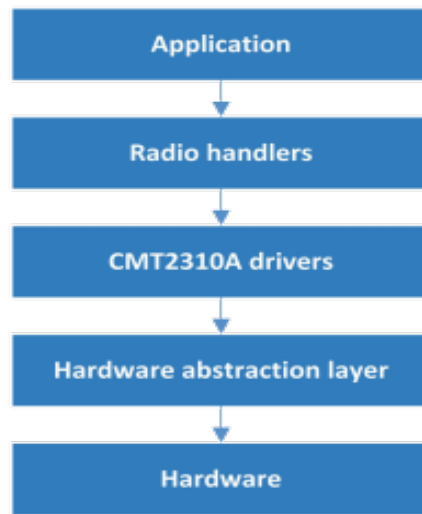


Figure 18. Software Hierarchy Architecture

4.2 Software Implementation and Calling Relation

In this section, it will introduce the calling relationship among modules, the system initialization process and working procedures, which are included in the following 5 files:

1. main.c : Application layer implementation
2. radio.c: Chip processing layer implementation
3. cmt2310a.c: Chip driver layer implementation
4. cmt2310a_hal.c : Hardware abstraction layer implementation
5. cmt_spi4.c: Chip SPI timing implementation, including accessing of register and FIFO timing.

The figure below shows the files of the first four layers and executed function of each layer. The file cmt_sp4.c provides the low-

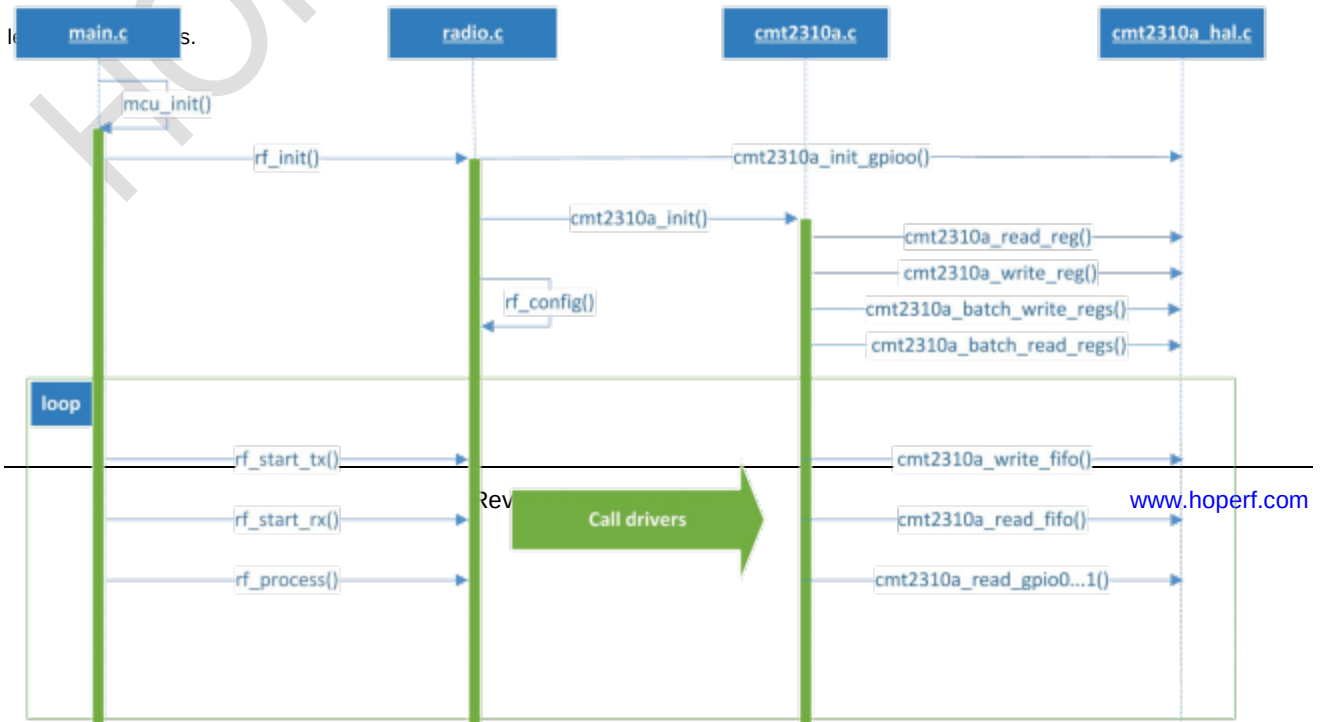


Figure 19. Software Implementation and Calling Relation

4.2.1 CMT2310A Initialization

After the chip is powered up, it must call the `rf_init ()` function and implement an initialization, which includes initializing SPI, GPIO0/1, NIRQ and selecting SPI mode (3 pin or 4 pin) soft reset, enabling or disabling some of the registers. The initialization procedure is shown in the figure below.

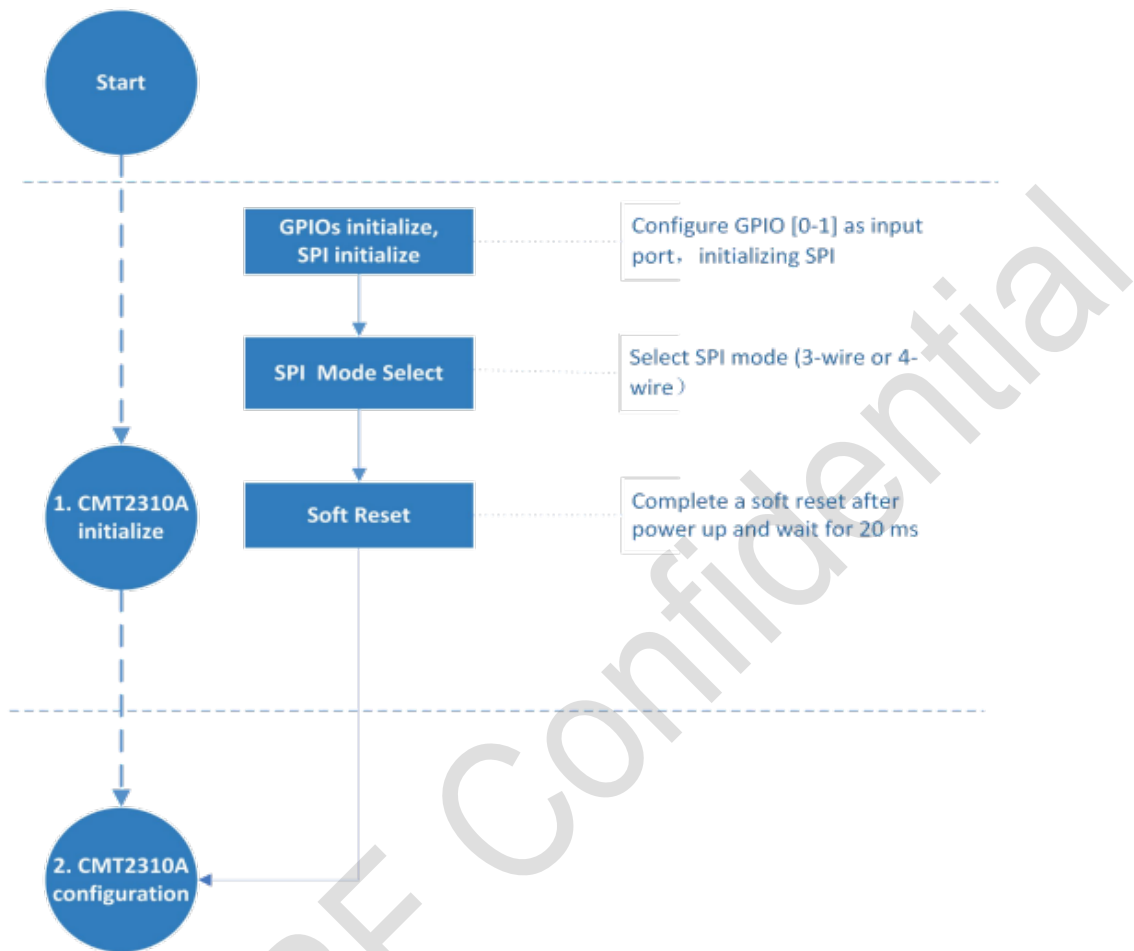


Figure 20. CMT2310A Initialization

4.2.2 CMT2310A Configuration

After the chip initialization, you need to call the `rf_config` function, configure the chip register under the IDLE state and then guide the firmware, enter into the READY state, do IR correction and finally configure the interrupt and GPIO. The following is the configuration flow chart:

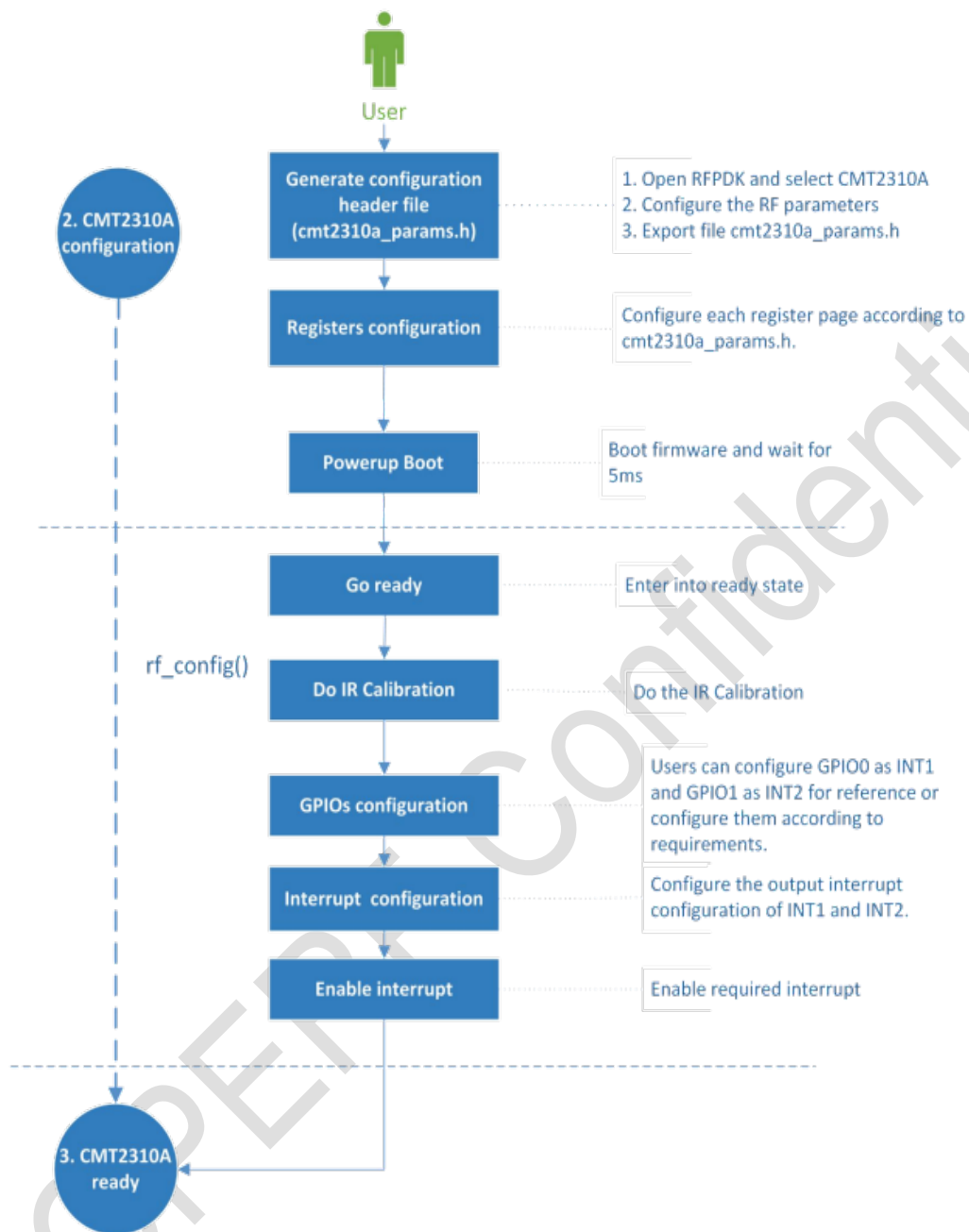


Figure 21. CMT2310A Configure Process

4.2.3 CMT2310A State Processing

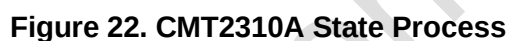
After the chip configuration is complete, you can call the `rf_start_tx()` or `rf_start_rx()` function to enter the transmit or receive state, and then cycle to call the `rf_process()` function for communication processing. Use a state machine in `rf_process()` to control the chip and return the status results to the application layer for processing.

rf_process() internal state:

1. RF_STATE_IDLE: Idle state
2. RF_STATE_RX_START: RX is in starting state, enabling FIFO reading and start receiving.
3. RF_STATE_RX_WAIT: RX is in waiting state, continuously detect the complete of interruption.
4. RF_STATE_RX_DONE: RX is in completion state, read FIFO, check and clear the interrupt.
5. RF_STATE_RX_TIMEOUT: RX is in timeout state, the chip needs to exit reception.
6. RF_STATE_TX_START: TX is in starting state, fill the FIFO data and start transmitting.
7. RF_STATE_TX_WAIT: TX is in completion state, check and clear the interrupt.
8. RF_STATE_TX_DONE: TX is in completion state, check and clear the interrupt
9. RF_STATE_TX_TIMEOUT: TX is in timeout state, the chip needs to exit transmission.
10. RF_STATE_ERROR: Error state. The chip cannot enter RX and TX state to soft reset the chip and re-configure it.

rf_process() returning result:

1. RF_IDLE: Chip is idle. Allow it to enter the RX or TX state.
2. RF_BUSY: Chip is busy. It is currently transmitting or receiving.
3. RF_RX_DONE: RX is completed. The upper layer can process the received data
4. RF_RX_TIMEOUT: RX is timeout.
5. RF_TX_DONE: RX is complete.
6. RF_TX_TIMEOUT: TX is timeout.
7. RF_ERROR: TX or RX is error.



Demo program is based on RFEB and developed by Keil5 IDE. It has a complete working directory:

- Rev 0.7 | Page 27 / 47

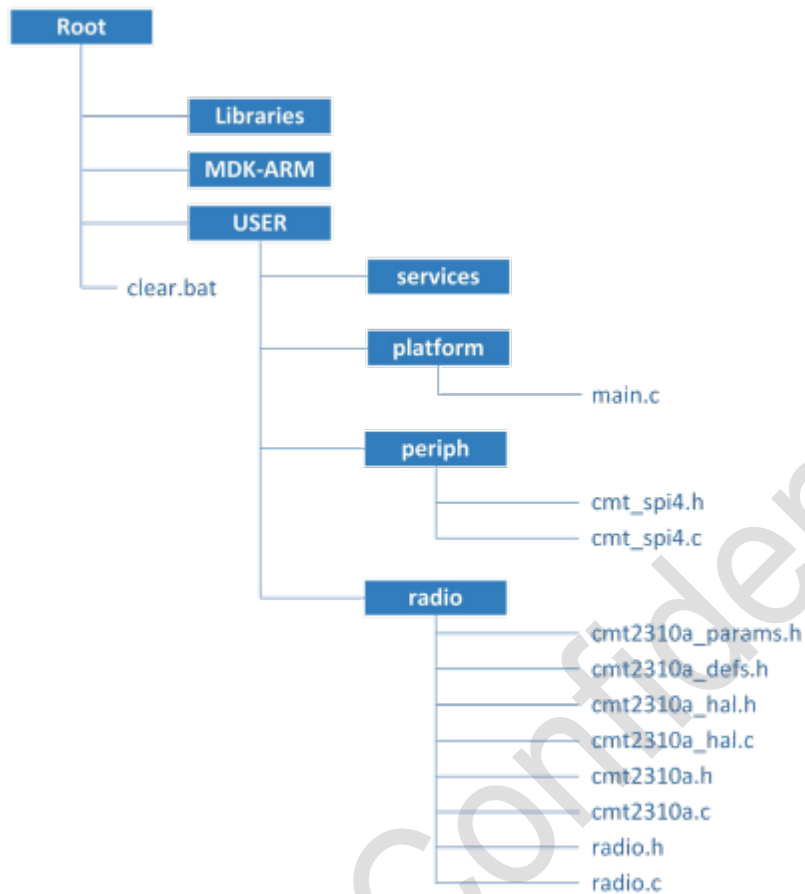


Figure 23. Software Structure Diagram

4.3.1 Application Layer Source Code

The application layer source code is main.c, which can select the master or slave by configuring g_enable_master, calling on_master() and on_slave() to transmitting and receiving.

```

#define RF_PACKET_SIZE 128          /* Define the payload size here */

static u8 g_rx_buffer [RF_PACKET_SIZE]; /* RF Rx buffer */
static u8 g_tx_buffer [RF_PACKET_SIZE]; /* RF Tx buffer */

static BOOL g_enable_master = FALSE; /* Master/Slave selection */

void mcu_init (void);

/* Manages the master operation */
void on_master (void);

/* Manages the slave operation */
void on_slave (void);
  
```

4.3.2 Simulating SPI Source Code

cmt_spi4.c is the SPI communication timing of CMT2310A and if there is a need to remove it to other MCU platform, some of the following macro-definition has to be modified:

```
/* *****  
* The following need to be modified by user  
* ***** */  
#define cmt_spi4_csb_out ()      SET_GPIO_OUT (CMT_CSB_GPIO)  
#define cmt_spi4_miso_out ()    SET_GPIO_OUT (CMT_MISO_GPIO)  
#define cmt_spi4_miso_in ()     SET_GPIO_IN (CMT_MISO_GPIO)  
#define cmt_spi4_scl_out ()     SET_GPIO_OUT (CMT_SCL_GPIO)  
#define cmt_spi4_mosi_out ()    SET_GPIO_OUT (CMT_MOSI_GPIO)  
#define cmt_spi4_mosi_in ()     SET_GPIO_IN (CMT_MOSI_GPIO)  
  
#define cmt_spi4_csb_1 ()       SET_GPIO_H (CMT_CSB_GPIO)  
#define cmt_spi4_csb_0 ()       SET_GPIO_L (CMT_CSB_GPIO)  
  
#define cmt_spi4_miso_1 ()      SET_GPIO_H (CMT_MISO_GPIO)  
#define cmt_spi4_miso_0 ()      SET_GPIO_L (CMT_MISO_GPIO)  
#define cmt_spi4_miso_read ()   READ_GPIO_PIN (CMT_MISO_GPIO)  
  
#define cmt_spi4_scl_1 ()       SET_GPIO_H (CMT_SCL_GPIO)  
#define cmt_spi4_scl_0 ()       SET_GPIO_L (CMT_SCL_GPIO)  
  
#define cmt_spi4_mosi_1 ()      SET_GPIO_H (CMT_MOSI_GPIO)  
#define cmt_spi4_mosi_0 ()      SET_GPIO_L (CMT_MOSI_GPIO)  
#define cmt_spi4_mosi_read ()   READ_GPIO_PIN (CMT_MOSI_GPIO)  
/* ***** */
```

4.3.3 Abstract Hardware Layer Source Code

cmt2310a_hal.c is the abstract hardware layer source code and implement the access interface of chip register, FIFO and GPIO.

```
/* ! *****  
* @name    cmt2310a_init_gpio  
* @desc    Initializes the cmt2310a interface GPIOs.  
* *****/  
void cmt2310a_init_gpio (void);  
  
/* ! *****  
* @name    cmt2310a_read_reg
```

```

* @desc    Read the cmt2310a register at the specified address.
* @param   addr: register address
* @return  Register value
* *****/
u8 cmt2310a_read_reg (u8 addr);

/* ! *****/
* @name    cmt2310a_write_reg
* @desc    Write the cmt2310a register at the specified address.
* @param   addr: register address
*          dat: register value
* *****/
void cmt2310a_write_reg (u8 addr, u8 dat);

/* ! *****/
* @name    cmt2310a_batch_read_regs
* @desc    Reads the contents of the cmt2310a regs (
           page0: register address start from 0x28,
           page1: register address start from 0x0).
* @param   buf: buffer where to copy the regs read data
*          len: number of bytes to be read from the regs
* *****/
void cmt2310a_batch_read_regs (u8 buf [], u16 len);

/* ! *****/
* @name    cmt2310a_batch_write_regs
* @desc    Writes the buffer contents to the cmt2310a regs (
           page0: register address start from 0x28,
           page1: register address start from 0x00).
* @param   buf: buffer containing data to be put on the regs
*          len: number of bytes to be written to the regs
* *****/
void cmt2310a_batch_write_regs (const u8 buf [], u16 len);

/* ! *****/
* @name    cmt2310a_read_fifo
* @desc    Reads the contents of the cmt2310a FIFO.
* @param   buf: buffer where to copy the FIFO read data
*          len: number of bytes to be read from the FIFO
* *****/
void cmt2310a_read_fifo (u8 buf [], u16 len);
/* ! *****/

```

```

* @name    cmt2310a_write_fifo
* @desc    Writes the buffer contents to the cmt2310a FIFO.
* @param   buf: buffer containing data to be put on the FIFO
*          len: number of bytes to be written to the FIFO
* *****/
void cmt2310a_write_fifo (const u8 buf [], u16 len);

```

If there is a need to remove the Demo program to other MCU platform, some of the macro-definition of cmt2310a_hal.h has to be modified:

```

/* *****/
* The following need to be modified by user
* *****/
#define cmt2310a_set_gpio0_in ()      SET_GPIO_IN (CMT_GPIO0_GPIO)
#define cmt2310a_set_gpio1_in ()      SET_GPIO_IN (CMT_GPIO1_GPIO)
#define cmt2310a_set_nirq_in ()       SET_GPIO_IN (CMT_NIRQ_GPIO)
#define cmt2310a_read_gpio0 ()        READ_GPIO_PIN (CMT_GPIO0_GPIO)
#define cmt2310a_read_gpio1 ()        READ_GPIO_PIN (CMT_GPIO1_GPIO)
#define cmt2310a_read_nirq ()         READ_GPIO_PIN (CMT_NIRQ_GPIO)
#define cmt2310a_delay_ms (ms)        system_delay_ms (ms)
#define cmt2310a_delay_us (us)        system_delay_us (us)
#define cmt2310a_get_tick_count ()    g_nSysTickCount
/* *****/

```

4.3.4 Chip Driver Layer Source Code

cmt2310a.c is the chip driver layer source code. It provides the chip state switching operations, interrupts, GPIO, FIFO operations, general register configuration and accessing, etc, which belongs to the firmware and is unrelated to the MCU. Users don't need to modify it.

```

/* ! *****/
* @name    cmt2310a_init
* @desc    Initialize chip status.
* *****/
void cmt2310a_init (void)
{
#ifdef SPI_3W_EN /* cmt_spi4.h */
    cmt2310a_select_spi_wire_mode (1);
#endif
    cmt2310a_soft_reset ();
}

```

```
cmt2310a_delay_ms (20);  
}
```

4.3.5 Chip Processing Layer Source Code

radio.c is the chip processing layer source code

```
/* RF state machine */  
typedef enum {  
    RF_STATE_IDLE = 0,  
  
    RF_STATE_RX_START,  
    RF_STATE_RX_WAIT,  
    RF_STATE_RX_DONE,  
    RF_STATE_RX_TIMEOUT,  
    RF_STATE_TX_START,  
    RF_STATE_TX_WAIT,  
    RF_STATE_TX_DONE,  
    RF_STATE_TX_TIMEOUT,  
    RF_STATE_ERROR,  
} EnumRFStatus;  
  
/* RF process function results */  
typedef enum {  
    RF_IDLE = 0,  
  
    RF_BUSY,  
    RF_RX_DONE,  
    RF_RX_TIMEOUT,  
    RF_TX_DONE,  
    RF_TX_TIMEOUT,  
    RF_ERROR,  
} EnumRFResult;  
  
// #define ENABLE_ANTENNA_SWITCH
```

5 Appendix

5.1 Appendix 1 : SPI Read/Write Sample Code

```
void cmt_spi4_send (u8 data8)
{
    u8 i;

    for (i=0; i<8; i++)
    {
        cmt_spi4_scl_0 ();

        /* Send byte on the rising edge of SCL */
        if (data8 & 0x80)
            cmt_spi4_mosi_1 ();
        else
            cmt_spi4_mosi_0 ();

        cmt_spi4_delay ();

        data8 <<= 1;
        cmt_spi4_scl_1 ();
        cmt_spi4_delay ();
    }
}

u8 cmt_spi4_recv (void)
{
    u8 i;
    u8 data8 = 0xFF;

    for (i=0; i<8; i++)
    {
        cmt_spi4_scl_0 ();
        cmt_spi4_delay ();
        data8 <<= 1;

        cmt_spi4_scl_1 ();

        /* Read byte on the rising edge of SCL */
    }
    #ifndef SPI_3W_EN
```

```

        if (cmt_spi4_miso_read ())
#else
        if (cmt_spi4_mosi_read ())
#endif

        data8 |= 0x01;
    else
        data8 &= ~0x01;

    cmt_spi4_delay ();
}

return data8;
}

void cmt_spi4_write (u8 addr, const u8* p_buf, u16 len)
{
    u16 i;

    cmt_spi4_mosi_1 ();
    cmt_spi4_mosi_out ();
    cmt_spi4_mosi_1 ();

    cmt_spi4_scl_0 ();
    cmt_spi4_scl_out ();
    cmt_spi4_scl_0 ();

    cmt_spi4_csb_0 ();
    cmt_spi4_csb_out ();
    cmt_spi4_csb_0 ();

    /* > 0.5 SCL cycle */
    cmt_spi4_delay ();
    cmt_spi4_delay ();

    /* r/w = 0 */
    cmt_spi4_send (addr&0x7F);

    for (i = 0; i < len; i++)
    {
        cmt_spi4_send (p_buf [i]);
    }
}

```

```
cmt spi4 scl 0 ();
```

HOPERF Confidential

--

HOPERF Confidential

```

    /* > 0.5 SCL cycle */
    cmt_spi4_delay ();
    cmt_spi4_delay ();

    cmt_spi4_csb_1 ();

    cmt_spi4_mosi_1 ();
#ifdef SPI_3W_EN
    cmt_spi4_mosi_in ();
#endif
}

void cmt_spi4_read (u8 addr, u8* p_buf, u16 len)
{
    u16 i;

    cmt_spi4_mosi_1 ();
    cmt_spi4_mosi_out ();
    cmt_spi4_mosi_1 ();
#ifdef SPI_3W_EN
    cmt_spi4_miso_1 ();
    cmt_spi4_miso_in ();
    cmt_spi4_miso_1 ();
#endif
    cmt_spi4_scl_0 ();
    cmt_spi4_scl_out ();
    cmt_spi4_scl_0 ();

    cmt_spi4_csb_0 ();
    cmt_spi4_csb_out ();
    cmt_spi4_csb_0 ();

    /* > 0.5 SCL cycle */
    cmt_spi4_delay ();
    cmt_spi4_delay ();

    /* r/w = 1 */
    cmt_spi4_send (addr | 0x80);
#ifdef SPI_3W_EN
    cmt_spi4_mosi_in ();
    cmt_spi4_delay ();

```

#endif

HOPERF Confidential

```
for (i = 0; i < len; i++)
{
    p_buf [i] = cmt_spi4_recv ();
}

cmt_spi4_scl_0 ();

/* > 0.5 SCL cycle */
cmt_spi4_delay ();
cmt_spi4_delay ();

cmt_spi4_csb_1 ();

cmt_spi4_mosi_1 ();

}
```

5.2 Appendix 2 : State Switching Library Function Sample Code


```

/* ! *****
* @name    cmt2310a_go_sleep
* @desc    Entry SLEEP mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_sleep (void)
{
    return cmt2310a_auto_switch_state (M_GO_SLEEP);
}

/* ! *****
* @name    cmt2310a_go_ready
* @desc    Entry ready mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_ready (void)
{
    return cmt2310a_auto_switch_state (M_GO_READY);
}

/* ! *****
* @name    cmt2310a_go_txfs

```

```

* @desc    Entry TFS mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_txfs (void)
{
    return cmt2310a_auto_switch_state (M_GO_TXFS);
}

/* ! *****/
* @name    cmt2310a_go_rxfs
* @desc    Entry RFS mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_rxfs (void)
{
    return cmt2310a_auto_switch_state (M_GO_RXFS);
}

/* ! *****/
* @name    cmt2310a_go_tx
* @desc    Entry Tx mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_tx (void)
{
    return cmt2310a_auto_switch_state (M_GO_TX);
}

/* ! *****/
* @name    cmt2310a_go_rx
* @desc    Entry Rx mode.
* @return  TRUE or FALSE
* *****/
BOOL cmt2310a_go_rx (void)
{
    return cmt2310a_auto_switch_state (M_GO_RX);
}

```

5.3 Appendix 3 : Initialization Function Sample Code

```
void rf_init (void)
{
    cmt2310a_init_gpio ();

    cmt2310a_init ();

    cmt2310a_go_sleep ();
    cmt2310a_delay_ms (2);

    rf_config ();
}

void rf_config (void)
{
    /* Config registers */
    cmt2310a_config_page_regs (0, g_cmt2310a_page0, CMT2310A_PAGE0_SIZE); /* config page 0 */
    cmt2310a_config_page_regs (1, g_cmt2310a_page1, CMT2310A_PAGE1_SIZE); /* config page 1 */
#ifdef CMT2310A_AUTO_RX_HOP_ENABLE /* cmt2310a_param.h exported by rfpdk */
    cmt2310a_select_reg_page (2);
    for (u8 idx = 0; idx < freq_times_val; idx++)
    {
        cmt2310a_write_reg (idx, g_cmt2310a_page2 [idx]); /* config page 2 */
    }
    cmt2310a_select_reg_page (0);
    cmt2310a_write_reg(CMT2310A_CTL_REG_12, freq_space_val);
    cmt2310a_write_reg(CMT2310A_CTL_REG_13, freq_times_val);
    cmt2310a_write_reg(CMT2310A_CTL_REG_14, freq_switch_state_val);
#endif
    cmt2310a_powerup_boot ();
    cmt2310a_delay_ms (5);

    cmt2310a_go_ready ();
    cmt2310a_delay_ms (2);

    /* do ir calibration */
    cmt2310a_api_command_and_wait (0x01);
    cmt2310a_api_command_and_wait (0x01);

    /* Config GPIOs */
    cmt2310a_config_gpio0 (GPIO0_SEL_INT1); /* INT1 -> GPIO0 */
    cmt2310a_config_gpio1 (GPIO1_SEL_INT2); /* INT2 -> GPIO1 */
}
```

```
/* Config interrupt */
cmt2310a_config_interrupt (
```

```
    INT_SRC_TX_DONE, /* Config TX_DONE -> INT1 */
    INT_SRC_PKT_DONE /* Config PKT_DONE -> INT2 */
);
```

```
/* config interrupt mode */
cmt2310a_config_interrupt_mode (INT1_TYPE_SEL_MODE1, INT2_TYPE_SEL_MODE1);
```

```
/* config interrupt polar */
cmt2310a_set_interrupt_polar (0,0); /* INT1: active-high, INT2: active-high */
```

```
/* enable interrupt source */
cmt2310a_enable_interrupt_source_0 (
```

```
    M_TX_DONE_EN      |
    M_PREAM_PASS_EN   |
    M_SYNC_PASS_EN     |
    M_ADDR_PASS_EN     |
    M_CRC_PASS_EN      |
    M_PKT_DONE_EN
);
```

```
/* Use a single 256-byte FIFO for either Tx or Rx */
//cmt2310a_enable_fifo_merge (1);
```

```
//cmt2310a_set_fifo_threshold (16);
```

```
/* save fifo at sleep state or not */
//cmt2310a_fifo_sleep_save (0); /* 0: save, 1: no save */
```

```
/* select pa mode, 0: single, 1: difference */
cmt2310a_select_pa_mode (0);
```

```
}
```

6 Revise History

Table 8. Revise History

Version No.	Chapter	Description	Date
0.5	All	Initial version	2020/09/ 17
0.6A	All	Review	2022/01/09
0.6B	All	1.The description of SCL is changed to SCLK; 2. Added the pin top view in Chapter 1.2; 3. 0x7B is the operation port of BRW in chapter 2.2; 4. The FIFO operation port is 0x7A in the figure in Chapter 2.3; 5. Modify the SPI timing in Chapter 2;	2022/01/19
0.6C	All	Review	2022/08/04
0.7	All	Review	2022/08/17

7 Contacts

Shenzhen Hope Microelectronics Co., Ltd.

Address: 30th floor of 8th Building, C Zone, Vanke Cloud City, Xili Sub-district, Nanshan, Shenzhen, GD, P.R. China

Tel: +86-755-82973805 / 4001-189-180

Fax: +86-755-82973550

Post Code: 518052

Sales: sales@hoperf.com

Website: www.hoperf.com

Copyright. Shenzhen Hope Microelectronics Co., Ltd. All rights are reserved.

The information furnished by HOPERF is believed to be accurate and reliable. However, no responsibility is assumed for inaccuracies and specifications within this document are subject to change without notice. The material contained herein is the exclusive property of HOPERF and shall not be distributed, reproduced, or disclosed in whole or in part without prior written permission of HOPERF. HOPERF products are not authorized for use as critical components in life support devices or systems without express written approval of HOPERF. The HOPERF logo is a registered trademark of Shenzhen Hope Microelectronics Co., Ltd. All other names are the property of their respective owners.

HOPERF Confidential